

Build Task — Config & Logging Foundation — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real codebase would actually write it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building two small files that work together: one that loads settings (a config loader), and one that sets up logging.

The config loader's job: read some values from the environment, and either hand back something you can trust, or stop with a clear error message if something needed is missing. Fail early — right at startup — not later when it's confusing.

The logger's job: one function, `get_logger(name)`, that any file can call to get a working logger. Python's built-in `logging` module already does most of the work.

Here are the exact pieces you need to look up and use:

- `os.getenv(key)` — gives you the value, or `None` if it isn't set.
- `load_dotenv()` from `python-dotenv` — call it once near the top, before reading any values. It loads your `.env` file's contents in.
- A simple class (or a `@dataclass`) to hold your config values together.
- `logging.getLogger(name)` — always gives back the *same* logger if you call it twice with the same name.
- `logging.StreamHandler()` — prints to the screen. `logging.FileHandler(path)` — writes to a file.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You're building two small, separate things that get used together: a **config loader** and a **logger factory**. Keep them as two separate files, doing two separate jobs.

Think about the config loader first. Its whole job is: read some environment variables, and either hand back an object you can trust, or fail loudly with a clear reason. The key idea is **fail at startup, not later** — if `OPENAI_API_KEY` is missing, you want to know *immediately*, with a message that names the missing key, not five minutes later as a confusing `AuthenticationError` from OpenAI.

For the logger, the idea is: one function, `get_logger(name)`, that any file can call to get a working logger. Don't overthink this one — Python's built-in `logging` module already does almost everything you need.

Here's what to actually go look at:

- `os.getenv(key)` — returns the value, or `None` if the key isn't set. This is your basic tool for reading environment variables. `os.environ[key]` is the alternative, but it raises `KeyError` instead of returning `None` — decide which behavior you want to build on top of.
- `python-dotenv`'s `load_dotenv()` — call this once, near the top of your program, before you read any environment variables. It reads `.env` and loads its contents into `os.environ` for you.
- A `dataclass` (`from dataclasses import dataclass`) is a clean way to build your `Config` object — it gives you a typed object with almost no boilerplate.
- `logging.getLogger(name)` always returns the *same* logger object if called twice with the same name — Python caches them for you. You don't need to build your own caching.
- `logging.StreamHandler()` writes to the console; `logging.FileHandler(path)` writes to a file. Each handler can have its own `.setLevel(...)`, independent of the logger's own level.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think past "read some env vars and raise if one is missing" — ask **who imports this config module, and what happens if `load_config()` is called more than once in the same program?** A chatbot script, a test file, and a Streamlit page might all import `config.py` and each call `load_config()` — do you want to re-read the `.env` file and re-validate every single time, or load it once and hand back the same trusted object everywhere?

The same question applies to `get_logger(name)` : if two different files both call `get_logger("my_app")` , Python's `logging` module already gives back the *same* logger object both times — but only if you don't accidentally attach a new handler each call. The real design question isn't "how do I read an env var," it's "how do I make sure this setup code is safe to call from many places without doing extra work or duplicating output."

Two extra pieces answer that design question:

- **`logger.handlers`** — a list. Checking `if not logger.handlers:` before attaching a new one is what makes `get_logger(name)` safe to call many times without duplicating console output — exactly the multi-caller problem above.
- **A module-level cache for config**, e.g. a single `_config: Config | None = None` variable at the top of `config.py` , checked at the start of `load_config()` — if it's already set, return it immediately instead of re-reading `.env` and re-validating. This is the same idea as `logger.handlers` , applied to config instead of logging: do the expensive/careful work once, reuse the result everywhere.

```
_config: "Config | None" = None

def load_config() -> "Config":
    global _config
    if _config is not None:
        return _config
    # ... do the real loading and validation here ...
    _config = Config(...)
    return _config
```

`global _config` is needed because you're *reassigning* the module-level variable from inside a function — without it, Python would treat `_config` as a new local variable instead.

Difference between Basic, Intermediate, and Advanced: Basic names the two files, their jobs, and the exact tools you'll need — `os.getenv` , `load_dotenv()` , a dataclass,

`logging.getLogger` , and the two handler types — in plain words. Intermediate explains what each of those tools actually does and why, plus the "fail at startup" principle with a concrete reason it matters. Advanced asks the harder design question underneath both files — not "how do I read an env var" but "how do I make this setup code safe to call from many places without doing extra work or duplicating output" — and adds a caching pattern (a `_config` cache, and `logger.handlers`) to both the config loader and the logger factory, which only matters once you imagine this code being imported from several files at once, as it will be in every later project.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
exceptions.py:
    make a MissingConfigError, it's a kind of Exception

config.py:
    make a Config holder with fields: openai_api_key, log_level

    function load_config():
        run load_dotenv() to read the .env file
        read OPENAI_API_KEY
        if it's missing or empty: raise MissingConfigError, name the missing key
        read LOG_LEVEL, use "INFO" if it isn't set
        return a Config built from these

logging_setup.py:
    function get_logger(name):
        get (or make) a logger for this name
        if it doesn't have a screen handler yet:
            make one, set its level from an env var (default INFO)
            attach it
        return the logger
```

The trickiest part of turning that plan into real code — checking a required key is really missing, not just empty:

```
import os

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError("Required environment variable is missing: " + key)
    return value
```

Expected output if you run just this (nothing calls `require_env` yet): nothing — defining a function doesn't run it. You need to add a call below this to see anything happen.

Use this same idea for every required key. For the logger's double-handler problem:

```
if not logger.handlers:  
    # only set up a handler the first time  
    ...
```

Try finishing the rest yourself before looking at the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

exceptions.py :

```
class MissingConfigError is an Exception
```

config.py :

```
define a Config data holder with fields: openai_api_key, log_level

function load_config():
    call load_dotenv() to read the .env file into the environment
    read OPENAI_API_KEY from the environment
    if it is missing or empty:
        raise MissingConfigError, message naming "OPENAI_API_KEY"
    read LOG_LEVEL from the environment, default to "INFO" if not set
    return a Config built from these values
```

logging_setup.py :

```
function get_logger(name):
    get (or create) a logger for this name
    if this logger doesn't already have a console handler attached:
        create a StreamHandler, set its level from an env var (default INFO)
        attach it to the logger
    set the logger's overall level low enough that the handler's level is what ac
    return the logger
```

The "if this logger doesn't already have a handler" check matters — without it, calling `get_logger("x")` twice would attach two handlers, and you'd see every log line printed twice.

Turning that plan into real code, here's the one piece worth seeing on its own first — this is *not* the full solution, just the trickiest part (checking a required key is genuinely missing, not just falsy):

```
import os

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
```

```
    raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value
```

Use this same pattern for every required key in `load_config()`, instead of writing the same `if` check out by hand each time.

For the logger's duplicate-handler problem, the check looks like this:

```
if not logger.handlers:
    # only attach a handler the first time this logger is configured
    ...
```

Try finishing the rest yourself before looking at the full Solution below.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
exceptions.py:
    class MissingConfigError is an Exception

config.py:
    module-level: _config = None    # cache, so load_config() only does real work

    function require_env(key):
        read key from the environment
        if missing or empty: raise MissingConfigError naming key
        return the value

    function load_config():
        if _config is already set: return it immediately (cache hit)
        call load_dotenv()
        api_key = require_env("OPENAI_API_KEY")
        log_level = read LOG_LEVEL, default "INFO"
        build Config, store it in _config, return it

logging_setup.py:
    function get_logger(name):
        logger = logging.getLogger(name)
        if logger.handlers is not empty: return logger immediately (already confi
        otherwise: build a StreamHandler with a real Formatter (timestamp, name,
        attach it, set the logger's level, return it
```

Notice both files now follow the *same* shape: check a cheap condition first ("is this already done?"), and only do the real work if the answer is no. That's the pattern from Hint 1's Advanced section, applied consistently across both files.

The caching piece from that pseudocode, made real — this is the one part that's genuinely easy to get wrong, so it's worth seeing on its own before the full Solution:

```
_config: "Config | None" = None

def load_config() -> "Config":
    global _config
    if _config is not None:
        return _config
```

```

load_dotenv()
api_key = require_env("OPENAI_API_KEY")
log_level = os.getenv("LOG_LEVEL", "INFO")

_config = Config(openai_api_key=api_key, log_level=log_level)
return _config

```

Expected output if you call `load_config()` twice in a row with a valid `.env`: nothing is printed by `load_config()` itself (it just returns a `Config` object) — but the second call skips `load_dotenv()` and `require_env()` entirely and returns the exact same object the first call built. You can prove this to yourself with `first_call is second_call` — it should print `True`.

Fill in `require_env` and the logger's handler check yourself, this time combined with this caching pattern, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic's pseudocode does the work every single time it's called, with no reuse, and its near-complete code proves the required-key check works at all. Intermediate is the same pseudocode shape described more precisely, plus the type contract real Python expects (`key: str -> str`) — still no caching. Advanced adds a cache check at the top of *both* functions — the config loader and the logger factory — so calling either one repeatedly from different files in a bigger program is cheap and safe instead of wasteful or bug-prone, and shows that caching made real and provably working (`first_call is second_call` → `True`).

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below shows the exact output you'd see if you ran it, right after the code, on a machine with a `.env` file containing `OPENAI_API_KEY=sk-test-123` (and no `LOG_LEVEL`, unless a block says otherwise). Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# exceptions.py
class MissingConfigError(Exception):
    pass
```

```
# config.py
import os
from dotenv import load_dotenv
from exceptions import MissingConfigError

class Config:
    def __init__(self, openai_api_key, log_level):
        self.openai_api_key = openai_api_key
        self.log_level = log_level

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError("Required environment variable is missing: " + key)
    return value

def load_config():
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(api_key, log_level)
```

```
# logging_setup.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# main.py – proving it works
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)
logger.info("Loaded config for " + config.openai_api_key)
```

Expected output:

```
Loaded config for sk-test-123
```

And with `OPENAI_API_KEY` missing from `.env` entirely:

```
Traceback (most recent call last):
...
exceptions.MissingConfigError: Required environment variable is missing: OPENAI_A
```

This version works correctly and meets every Build Task requirement. It's missing type hints, and it builds error messages with `+` instead of an f-string — both fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a dataclass-based config

exceptions.py

```
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

config.py

```
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value

def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

logging_setup.py

```

import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)

    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)

        handler = logging.StreamHandler()
        handler.setLevel(level)
        formatter = logging.Formatter("%(asctime)s %(name)s %(levelname)s %(message)s")
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        logger.setLevel(level)

    return logger

```

```

# main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)
logger.info(f"Loaded config for {config.openai_api_key}")

```

Expected output (the exact timestamp will differ on your machine — `%(asctime)s` always prints the current time):

```
2026-09-10 09:00:00,123 __main__ INFO Loaded config for sk-test-123
```

Why this approach: a `@dataclass` gives you a typed, readable `Config` object with almost no extra code. This is a very common, standard pattern in real Python projects.

Approach 2 — a plain class, with file logging added

This version uses a plain class instead of a dataclass (more explicit, a bit more typing), and adds a file handler alongside the console one, since some projects want both.

`config.py`

```

import os
from dotenv import load_dotenv
from exceptions import MissingConfigError

class Config:
    def __init__(self, openai_api_key: str, log_level: str) -> None:
        self.openai_api_key = openai_api_key
        self.log_level = log_level

def load_config() -> Config:
    load_dotenv()

    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise MissingConfigError("Required environment variable is missing: OPENAI_API_KEY")

    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)

```

logging_setup.py

```

import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)

    if logger.handlers:
        return logger

    console_level_name = os.getenv("LOG_LEVEL", "INFO")
    console_level = getattr(logging, console_level_name.upper(), logging.INFO)

    console_handler = logging.StreamHandler()
    console_handler.setLevel(console_level)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)

    formatter = logging.Formatter("%(asctime)s %(name)s %(levelname)s %(message)s")
    console_handler.setFormatter(formatter)
    file_handler.setFormatter(formatter)

```

```

logger.addHandler(console_handler)
logger.addHandler(file_handler)
logger.setLevel(logging.DEBUG)

return logger

```

```

# main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)
logger.debug("Starting up")
logger.info(f"Loaded config for {config.openai_api_key}")

```

Expected output on screen (console handler is at `INFO` , so the `DEBUG` line is filtered out and never reaches the screen):

```

2026-09-10 09:00:00,456 __main__ INFO Loaded config for sk-test-123

```

Expected content appended to `app.log` (file handler is at `DEBUG` , so both lines land there):

```

2026-09-10 09:00:00,321 __main__ DEBUG Starting up
2026-09-10 09:00:00,456 __main__ INFO Loaded config for sk-test-123

```

Difference from Basic: both Intermediate approaches add full type hints — `openai_api_key: str` , -> `Config` , -> `logging.Logger` — turning every signature into documentation a reader (or an editor's autocomplete) can rely on without opening the function body. Both also add a real `Formatter` , so log lines carry a timestamp, logger name, and level instead of just the bare message. Approach 2 additionally adds a `FileHandler` , so you keep a full `DEBUG` -level record in `app.log` even while the screen only shows `INFO` and above — closer to what a real deployed script needs, since you can't always watch the screen live.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — dataclass + file handler + a config cache (the Hint 2 pattern, completed)

exceptions.py

```
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

config.py

```
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value

_config: "Config | None" = None

def load_config() -> Config:
    global _config
    if _config is not None:
        return _config

    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
```

```
log_level = os.getenv("LOG_LEVEL", "INFO")

_config = Config(openai_api_key=api_key, log_level=log_level)
return _config
```

logging_setup.py

```
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    formatter = logging.Formatter("%(asctime)s %(name)s %(levelname)s %(message)s")

    console_level_name = os.getenv("LOG_LEVEL", "INFO")
    console_level = getattr(logging, console_level_name.upper(), logging.INFO)
    console_handler = logging.StreamHandler()
    console_handler.setLevel(console_level)
    console_handler.setFormatter(formatter)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(formatter)

    logger.addHandler(console_handler)
    logger.addHandler(file_handler)
    logger.setLevel(logging.DEBUG)
    return logger
```

```
# main.py
from config import load_config

first = load_config()
second = load_config()
print(first is second)
```

Expected output:

```
True
```

The second `load_config()` call skips `load_dotenv()` and `require_env()` entirely — it returns the cached `Config` object from the first call.

Approach 2 — `pydantic-settings`, validation declared instead of hand-written

Instead of writing `require_env()` yourself, you describe your config's shape, and a library reads and validates the environment for you.

`config.py`

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Config(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")

    openai_api_key: str
    log_level: str = "INFO"

def load_config() -> Config:
    return Config()
```

There's no `exceptions.py` needed for the missing-key case here — `BaseSettings` reads `.env` and the real environment automatically (no separate `load_dotenv()` call), and if `openai_api_key` isn't set anywhere, creating `Config()` raises `pydantic_core.ValidationError` on its own, without you writing a check for it.

```
# main.py, with OPENAI_API_KEY missing from .env
from config import load_config

config = load_config()
```

Expected output:

```
Traceback (most recent call last):
...
pydantic_core._pydantic_core.ValidationError: 1 validation error for Config
openai_api_key
  Field required [type=missing, input_value={}, input_type=dict]
```

And with `OPENAI_API_KEY=sk-test-123` present:

```
config = load_config()
print(config.openai_api_key)
```

```
sk-test-123
```

Approach 3 — `get_logger` with `dictConfig` -style setup, and `Config` that validates `LOG_LEVEL` is real

The first two Advanced approaches trust that whatever string is in `LOG_LEVEL` is a real logging level — `getattr(logging, "INFO", logging.INFO)` silently falls back to `INFO` for *any* typo (`"INFO0"`, `"debug"` lowercase without `.upper()` handled right, etc.), which hides mistakes instead of catching them. This approach catches that at config-load time, and configures logging from one settings dictionary instead of building handlers by hand.

`exceptions.py`

```
class MissingConfigError(Exception):
    pass

class InvalidConfigError(Exception):
    """Raised when a setting is present but holds an invalid value."""
    pass
```

`config.py`

```
import logging
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from exceptions import MissingConfigError, InvalidConfigError

VALID_LOG_LEVELS = ("DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL")

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value

@dataclass
```

```

class Config:
    openai_api_key: str
    log_level: str

def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")

    log_level = os.getenv("LOG_LEVEL", "INFO").upper()
    if log_level not in VALID_LOG_LEVELS:
        raise InvalidConfigError(
            f"LOG_LEVEL must be one of {VALID_LOG_LEVELS}, got: {log_level!r}"
        )

    return Config(openai_api_key=api_key, log_level=log_level)

```

logging_setup.py

```

import logging
import logging.config

def configure_logging(log_level: str) -> None:
    settings = {
        "version": 1,
        "disable_existing_loggers": False,
        "formatters": {
            "default": {"format": "%(asctime)s %(name)s %(levelname)s %(message)s"},
        },
        "handlers": {
            "console": {
                "class": "logging.StreamHandler",
                "level": log_level,
                "formatter": "default",
            },
        },
        "root": {"level": log_level, "handlers": ["console"]},
    }
    logging.config.dictConfig(settings)

def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)

```

```
# main.py, with a bad value: LOG_LEVEL=VERBOSE in .env
from config import load_config

config = load_config()
```

Expected output:

```
Traceback (most recent call last):
...
exceptions.InvalidConfigError: LOG_LEVEL must be one of ('DEBUG', 'INFO', 'WARNIN
```

And with a valid `.env` (`LOG_LEVEL=INFO` or unset):

```
# main.py
from config import load_config
from logging_setup import configure_logging, get_logger

config = load_config()
configure_logging(config.log_level)
logger = get_logger(__name__)
logger.info(f"Loaded config for {config.openai_api_key}")
```

Expected output:

```
2026-09-10 09:00:00,789 __main__ INFO Loaded config for sk-test-123
```

Here `configure_logging()` is called once, at startup, with the already-validated `log_level` — after that, any file can just call `get_logger(__name__)` with no setup logic of its own, since `dictConfig` already wired the root logger and its handler.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate's `load_config()` and `get_logger()` do real, correct work, but redo it on every single call, and never check that `LOG_LEVEL` is an actual logging level name. Approach 1 fixes the "redo it every call" problem with a module-level cache — cheap, dependency-free, and the most direct upgrade from Intermediate. Approach 2 replaces hand-written validation with a library (`pydantic-settings`) that declares the config's shape and lets the library raise a detailed error on any missing or wrong-typed field — less code to maintain, at the cost of a new dependency and a less custom error message. Approach 3 tackles a different gap entirely: it validates that `LOG_LEVEL` is genuinely one of Python's real level names (catching a typo like `"VERBOSE"` at startup instead of it silently becoming `INFO`), and switches `get_logger()` to `dictConfig` , which centralizes every handler/formatter/level setting in one dictionary instead of building `StreamHandler` objects by hand in every file that needs one.

Which one should you actually use? For the Build Task as written, Intermediate Approach 1 already satisfies every requirement and constraint — it's what most people should ship first. Reach for Advanced Approach 1's caching once you notice `load_config()` genuinely gets called from several files in the same run (which, per this document's Goal, will happen the moment you reuse these files in your next project). Reach for Advanced Approach 2 (`pydantic-settings`) once your config grows past 2-3 fields, or once you want validation like "must be a valid URL" or "must be a number in this range" without writing it by hand. Reach for Advanced Approach 3's `LOG_LEVEL` validation and `dictConfig` any time a typo in an env var has actually bitten you, or once you're configuring logging for more than one handler and want it in one readable place instead of scattered `if` checks. None of these are wrong — they're solving progressively less common problems, so match the approach to the problem you actually have, not the fanciest one available.